



Actividad práctica en parejas

Objetivo: tomar el conversor de la clase pasada, que ahora guarda un historial de conversiones y lo muestra en una segunda ventana, y pasar de un controlador que actualiza las vistas a mano a un modelo que avisa a sus observadoras. Después agregar una tercera ventana sin tocar el controlador.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/P00-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es 12-observer/actividad.

Es el conversor de pesos a dólares, euros y reales, repartido en `modelo`, `vista` y `controlador`, con dos agregados: el `Conversor` guarda cada conversión en un historial, y `VistaHistorial` lo muestra en una `JTable`. `ModeloTablaConversiones`, el `TableModel` de esa tabla, ya está completo y no se modifica. `Programa.java` no se modifica salvo comentar y descomentar los bloques que indica cada parte.

Parte 1: ver el problema

`ControladorConversor` convierte y después actualiza las dos ventanas a mano: `vista.mostrarResultado()` y `historial.actualizar()`, en cada uno de los tres métodos. En `convertirAEuros()` falta la segunda llamada. Se comprueba así:

1. Ejecutar `Programa` y convertir 1450 a dólares. El resultado aparece en la ventana principal y la fila aparece en la tabla.
2. Convertir 1450 a euros. El resultado aparece, la fila no. El historial del modelo sí tiene la conversión: al convertir a reales, la tabla muestra las tres filas de golpe.
3. Leer `ControladorConversor.java`. Cada vista es un parámetro más del constructor y una línea más en cada método, y una línea que falta es una ventana que muestra datos viejos.

Parte 2: el modelo avisa

Un TODO por vez, en orden. Cada TODO indica en qué archivo se trabaja. Entre el TODO 3 y el TODO 4 el proyecto no compila: es esperable.



- TODO 1 (`Conversor.java`): el `Conversor` acepta observadores con `agregarObservador()` y los avisa después de cada conversión, con `conversorCambio()`. La interfaz `ObservadorConversor` ya está en el paquete `modelo`.
- TODO 2 (`VistaConversor.java`): la vista implementa `ObservadorConversor` y muestra la última conversión, `conversor.getUltima()`, cuando el `Conversor` avisa.
- TODO 3 (`VistaHistorial.java`): la vista implementa `ObservadorConversor`; `conversorCambio()` hace lo que hacía `actualizar()`, que se borra.
- TODO 4 (`ControladorConversor.java`): el controlador deja de conocer a `VistaHistorial` y de actualizar vistas; cada método solo llama al `Conversor`. Pasar `Programa` al bloque de la parte 2, que registra las dos vistas como observadoras.

Con los cuatro TODO hechos, las dos ventanas muestran lo mismo después de cada conversión, euros incluidos.

Parte 3: la tercera ventana

- TODO 5: agregar `VistaContador`, una ventana con una etiqueta que dice cuántas conversiones se hicieron y cuántos pesos se convirtieron en total, sin modificar el controlador. Pasar `Programa` al bloque de la parte 3, con su `import`.

Antes de seguir, comprobar que las tres ventanas coinciden después de cada conversión, y que `ControladorConversor.java` no cambió desde el TODO 4.

Para discutir en la pareja y anotar:

- Después del TODO 4, ¿quién escribe en el `Conversor` y quién lee de él? ¿Alguna vista llama a un método que cambie el modelo?
- El `Conversor` no importa nada de `javax.swing` y la interfaz `ObservadorConversor` está en el paquete `modelo`. ¿Por qué la define el modelo y no la vista?
- Si `Programa` registra a `VistaHistorial` dos veces, ¿qué pasa en cada conversión? ¿Y si no la registra?
- `VistaHistorial` llama a `fireTableDataChanged()` y la `JTable` se redibuja. ¿Quién observa a quién en ese par?
- Un `JButton` con `addActionListener()`, ¿es el mismo patrón? ¿Quién es el observado y quién el observador?
- En el proyecto integrador, la partida es el modelo. ¿Qué ventanas la observarían?